第4周 SDD 加餐：Superpowers 详解 + 实...

> 本节加餐内容由咖哥通过AI的辅助从Superpowers官方资料中学习整理。同时提供了基于于知识库项目的Greenfield / Brownfield 两条路径的实操方向（目前我自己还**未在**课程环境逐步实测）。
>
> 大家可以挑一条上手尝试，遇到问题的话，可以通过AI工具排查或群里讨论来解决。

咱们现在已经会用 OpenClaw 写自己的 Skill，但 OpenClaw 的 Skill 是**消息网关上的**——只在 messaging profile 里跑，跟你写代码用的 Claude Code / OpenCode / Cursor / Codex 是两个世界。

Superpowers 是把“Skill 化“路推到**写代码侧**的工程方法论：

- 让 Claude Code（或 OpenCode / Codex / Gemini / Cursor / Copilot）有一套**统一的可复用 skill 库**
- 涵盖**软件开发全流程**：brainstorm → 写 plan → 拆任务 → 写代码 → 调试 → review
- **由 Jesse Vincent (obra) 创建并开源**，2026-01-15 被收进 Anthropic 官方 Claude Code marketplace —— GitHub 40.9k stars / 3.1k forks（写作时数据），已成事实标准

跟 OpenClaw skill 的区别如下：

```
                 Skill 描述（SKILL.md · 同一个标准）
                            ↓
            ┌──────────────────────────────┐
            ↓                              ↓
  OpenClaw 跑在消息网关侧                  Superpowers 跑在编码侧
  (Telegram / 飞书 / 微信)                 (Claude Code / OpenCode / Curs
  你在 13-15 节写过的就是这种              帮你写代码的"Claude 助手"用这种
```

上面的意思是说Skill 标准本身是开放的，同一个 SKILL.md 在两边都能跑。OpenClaw的Skills给运营助手看。我们SDD加餐之前学习的OpenSpec（或者Spec-Kit），BMAD，以及Superpowers 给的都是**编码侧的精装套装。**

Superpowers ≈ SDD 日常版

Superpowers 的核心信念：**AI 不是魔法盒，是有边界的工具。明确告诉它流程，比放任它自由更可靠。**

这跟 Week 1 SDD 那套“先 spec 后 code”异曲同工，**Superpowers 就是 SDD 思路在日常编码里的精装实现。**

你把 brainstorm（探索）→ writing-plans（拆任务）→ executing-plans（按步执行）这条链子在脑里映射回 SDD：spec → plan → tasks → implement，一一对应。

§1 安装

Superpowers 是**多平台**的 skill 集合，每个平台装法稍有差异。挑一个你日常用的：

平台	安装命令 / 文档
Claude Code	`/plugin install superpowers@claude-plugins-official`（在 Claude Code 里直接发）
OpenCode	编辑 `~/.config/opencode/opencode.json`，加 `"plugins": ["obra/superpowers"]`
跨工具通用 · `npx skills`	`npx skills install obra/superpowers` —— 装到 `~/.agents/skills/`，多工具共享
Codex CLI / Gemini CLI / Cursor / Copilot CLI	各家有专门的接入方式，见 obra/superpowers README

真实路径速查表

```
~/.agents/skills/<skill>/SKILL.md          ← 跨工具通用 · npx skills CLI 默
~/.config/opencode/skills/<skill>/SKILL.md ← OpenCode 全局（显式声明 non-u
~/.claude/skills/<skill>/SKILL.md          ← Claude Code 全局
./.opencode/skills/<skill>/SKILL.md        ← OpenCode 项目级
./.claude/skills/<skill>/SKILL.md          ← Claude Code 项目级
```

§2 核心 skills 详解

仓库 obra/superpowers/skills/ 下一共 14 个 skill。**主线 6 + 辅助 2 共 8 个就够覆盖日常。**一个一个简单说说，梳理知识脉络。

§2.1 `brainstorming` 思考前置 不让你立刻动手

官方 description：

> "You MUST use this **before any creative work** — creating features, building components, adding functionality, or modifying behavior. Explores user intent, requirements and design before implementation."

核心信念：

- `<HARD-GATE>` · **Do NOT invoke any implementation skill, write any code, scaffold any project, or take any implementation action until you have presented a design and the user has approved it.**
- 所有项目都走这个流程，不存在“这个太简单了不需要设计”

Anti-Pattern 反模式：

> "**This Is Too Simple To Need A Design**" —— "Simple" projects are where unexamined assumptions cause the most wasted work.
> 意思是，别觉得太简单的应用就不需要好好设计。作者认为都需要好好设计，所以要头脑好好的风暴风暴。——**佳哥不是很认可这个观点。因为我最不喜欢过度设计。**

9 步 checklist：

```
1. Explore project context (读文件 / 看 docs / 翻最近 commit)
2. Offer Visual Companion (如有视觉问题 · 单独发一条 message)
3. Ask clarifying questions (一次问一个 · 不批量)
4. Propose 2-3 approaches (含 trade-off + 推荐)
5. Present design (按复杂度分段 · 每段单独得到批准)
6. Write design doc → docs/superpowers/specs/YYYY-MM-DD-<topic>-design.m
7. Spec self-review (查 placeholder / 矛盾 / 模糊 / scope)
8. User reviews written spec (你看完文件再走下一步)
9. Transition to implementation → 调用 writing-plans
```

它解决的问题是，用户说“做个 todo app “，AI 立刻 `mkdir todo-app && touch app.py` 开干 —— **这是 Claude/GPT 默认行为，有可能大方向是错的。**brainstorming 强行把这个“立刻动手”动作锁住，逼你先讨论。

输出：一份 `design.md` ，**终点是调** `writing-plans` **，不会调任何其他实现 skill。**

§2.2 `writing-plans` 把 design 拆成可执行任务

官方 description：

> Use when you have a spec or requirements for a multi-step task, before touching code.

核心信念：

> Assume the engineer has **zero context for our codebase** and **questionable taste**. Document everything they need to know.

写 plan 时假设接手人是空白的，连测试设计都不会。看似贬低，实际上**这是 plan 质量保证。**

关键设计原则：

```
DRY · YAGNI · TDD · Frequent commits

文件结构在写任务前先 map 出来:
- 边界清晰、接口定义好
- 单一职责
- 一起改的文件放一起 (按职责拆 · 不按技术层拆)
- 现有代码库 follow established pattern
```

Bite-sized 任务粒度（每步 2-5 分钟）：

```
- [ ] Step 1: Write the failing test
- [ ] Step 2: Run it to make sure it fails
```

```
- [ ] Step 3: Implement minimal code to make it pass
- [ ] Step 4: Run tests, all pass
- [ ] Step 5: Commit
```

plan header 模板：

```
# [Feature Name] Implementation Plan

> **For agentic workers:** REQUIRED SUB-SKILL: Use superpowers:subagent-
> (recommended) or superpowers:executing-plans to implement this plan ta
> Steps use checkbox (`- [ ]`) syntax for tracking.

**Goal:** [一句话]
**Architecture:** [2-3 句]
**Tech Stack:** [关键技术]
```

输出：一份 `docs/superpowers/plans/YYYY-MM-DD-<feature-name>.md`

洞察：plan 文件 header 强制声明“用哪个 skill 来执行”，这把“plan 怎么执行“在文件层面就钉死了，不靠人记忆。这种**自描述文档** + 链式 skill 调用是 Superpowers 的核心模式。

§2.3 `executing-plans` 严格按 plan 跑、不跳步

官方 description：

> "Use when you have a written implementation plan to execute in a separate session with review checkpoints"

3 步流程：

```
Step 1 · Load and Review Plan
  - Read plan file
  - Review critically · 列出 concerns
  - 有疑问先跟 partner 沟通,无疑问 → 创建 TodoWrite 开干

Step 2 · Execute Tasks (for each task)
  - 标 in_progress
  - 按步骤精确执行 (每步 2-5 分钟)
  - 跑 verification (lint / test / 手动)
  - 标 completed

Step 3 · Complete Development
  - 全部 task 完成后,调 finishing-a-development-branch
```

STOP 立即执行的 4 种情况：

1. 撞到 blocker (缺依赖 / 测试挂 / 指令不清)
2. plan 有关键 gap 没法开始
3. 不理解某步指令
4. 验证反复失败

关键设计：

> Tell your human partner that Superpowers works **much better with access to subagents**. The quality will be significantly higher if run on a platform with subagent support (such as Claude Code or Codex). If subagents are available, use **subagent-driven-development** instead of this skill.

Superpowers 自己承认：单 session 跑不如 subagent 隔离跑。这是为什么 §2.7 的 `subagent-driven-development` 是更高级形态。

> 洞察：**"Don't force through blockers — stop and ask"** AI 默认会硬撑过去（试试 catch 错误、试试改个变量名让测试过），明确写上这条 SKILL 才能压住。

§2.4 `systematic-debugging` 找根因，不打补丁

官方 description：

> Use when encountering any bug, test failure, or unexpected behavior, **before proposing fixes**.

The Iron Law：

```
NO FIXES WITHOUT ROOT CAUSE INVESTIGATION FIRST
```

Phase 1　Root Cause Investigation 4 个动作：

1. **Read Error Messages Carefully** 不要跳过任何错误/警告、完整读 stack trace、记下行号文件路径错误码
2. **Reproduce Consistently** 能稳定触发吗？步骤是什么？每次都发生？不能复现 → 多收数据，**别猜**
3. **Check Recent Changes** ，包括git diff、最近 commit、新依赖、配置变更、环境差异
4. **Gather Evidence in Multi-Component Systems** 在每个组件边界加 instrumentation、log 数据进/出

核心信念：

- **Symptom fixes are failure**
- Random fixes waste time and create new bugs
- Quick patches mask underlying issues

特别适合用在以下场合（这些是 AI 最容易跳过 root cause 的情境）：

- 时间紧（紧急让 AI 想猜）
- 就这一个快速修复，看起来 obvious
- 已经试过多个修复
- 上一个修复没生效
- 你没完全搞懂这个问题

反模式禁忌：

> "Don't skip when issue seems simple — simple bugs have root causes too. You're in a hurry — rushing guarantees rework. Manager wants it fixed NOW — systematic is faster than thrashing."

快也是慢，慢就是快。猜着改 3 次的时间够正经查 1 次根因。

> 洞察：把 Iron Law 写成单行 ASCII 框框 + “Phase 1 没完成不能提 fix”的硬约束，这种**程序化的纪律**比“建议你认真排查”有用 100 倍。

---

§2.5 `test-driven-development` 先写失败测试再写实现

官方 description：

> Use when implementing any feature or bugfix, before writing implementation code

The Iron Law：

```
NO PRODUCTION CODE WITHOUT A FAILING TEST FIRST
```

最严苛的反模式禁令（原文照抄）：

> "Write code before the test? **Delete it. Start over.**"
> "**No exceptions:**
> Don't keep it as 'reference'
> Don't 'adapt' it while writing tests
> Don't look at it
> **Delete means delete**"
> "Implement fresh from tests. Period."

Superpowers 对“测试后置”是零容忍，这种强度在普通编码规范里很少见。

Red-Green-Refactor 完整闭环：

```
RED       · Write failing test (一个最小测试 · 表达期望行为)
  ↓
verify    · 测试真的失败了吗？是因为正确原因失败的吗？(不是语法错那种)
  ↓
GREEN     · 写最小的实现让测试过 (够用就行 · 不要超前实现)
  ↓
verify    · 全部测试都过了吗？
  ↓
REFACTOR  · 清理代码 (保持 green · 不引入新行为)
  ↓
NEXT      · 进入下一个 RED
```

Always 适用 / 例外要问 partner：

- Always: 新功能 / bug 修复 / refactor / 行为变更
- 例外（必须问）: 一次性原型 / 生成代码 / 配置文件
- **"Thinking 'skip TDD just this once'? Stop. That's rationalization."**

> 洞察：“**Verify fails correctly**”是一个易忽略的关键步骤， 测试可能因为 import 错 / 拼写错 / 框架问题失败 ，这类失败不算 RED，必须确认是“因为功能没实现”才失败的，才能进 GREEN。

§2.6 `requesting-code-review` + `receiving-code-review` • review 拆成两个 skill

为什么拆两个：发 review 请求 和 接 review 反馈是两种不同的纪律 · 各有各的反模式。

`requesting-code-review` 怎么发 review

核心机制：

> "Dispatch a code reviewer subagent. The reviewer gets **precisely crafted context for evaluation — never your session's history**. This keeps the reviewer focused on the work product, not your thought process, and **preserves your own context** for continued work."

不是主 agent 自己 review，是**派一个 fresh subagent 去 review。**
触
发时机

- Mandatory ：每个 task 完成后 / major feature 完成后 / merge 到 main 之前
- Optional：卡住了想要 fresh perspective / refactor 前要 baseline / 修完复杂 bug 之后

3 个分级反馈：

- **Critical** · 立即修
- **Important** · 继续之前必须修
- **Minor** · 留着以后

`receiving-code-review` 怎么接 review

核心信念：

> "Code review requires **technical evaluation**, not **emotional performance**." "Verify before implementing. Ask before assuming. Technical correctness over social comfort."

FORBIDDEN 禁止说的话：

```
NEVER:
- "You're absolutely right!"  (explicit CLAUDE.md violation)
- "Great point!" / "Excellent feedback!"  (performative)
- "Let me implement that now"  (before verification)

INSTEAD:
- Restate the technical requirement
- Ask clarifying questions
- Push back with technical reasoning if wrong
- Just start working (actions > words)
```

6 步响应模式：

```
1. READ      · 读完整反馈,不立即反应
2. UNDERSTAND · 用自己的话复述需求 (或主动问)
3. VERIFY    · 对照代码库现实 check
4. EVALUATE  · 这个建议对当前 codebase 真的合理吗?
```

```
5. RESPOND   · 技术性确认 或 reasoned pushback
6. IMPLEMENT · 一项一项做 · 每项测试
```

关键反模式：partner 说"修 1-6"，你只懂 1/2/3/6 不懂 4/5 怎么办？

```
❌ WRONG: 先把懂的 1/2/3/6 做了,4/5 等等再问
✅ RIGHT: 「我理解 1/2/3/6,关于 4 和 5 需要澄清后再继续」

WHY: 项目可能相互关联 · 部分理解 = 错误实现
```

> 洞察：把 review 当独立任务发一个干净的 subagent，避免“自己 review 自己”的盲区。

§2.7 `subagent-driven-development` 高级形态 任务一 subagent + 两阶段 review

官方 description：

> "Use when executing implementation plans with independent tasks in the current session"

核心机制：

> "Fresh subagent per task + two-stage review (spec then quality) = high quality, fast iteration"
> "Continuous execution: **Do not pause to check in with your human partner between tasks**. Execute all tasks from the plan without stopping. The only reasons to stop are: BLOCKED status you cannot resolve, ambiguity that genuinely prevents progress, or all tasks complete. **'Should I continue?' prompts and progress summaries waste their time** — they asked you to execute the plan, so execute it."

这条专治“AI 每做完一步都来问你 OK 吗”的烦人行为。

每个 task 走 4 步：

```
1. 派 implementer subagent (./implementer-prompt.md)
   → 它会问问题 / 实现 / 测试 / commit / self-review

2. 派 spec reviewer subagent (./spec-reviewer-prompt.md)
   → 验证代码符合 spec
   → 不符合 → implementer 修 spec gap → 重新 review

3. 派 code quality reviewer subagent (./code-quality-reviewer-prompt.md)
   → 验证代码质量
   → 不通过 → implementer 修质量问题 → 重新 review

4. 标 task 完成,进入下一个
```

vs `executing-plans` 的区别：

维度	executing-plans	subagent-driven-development
Session	同 session 跑	同 session · 但每 task fresh subagent
Context	累积 · 容易污染	隔离 · 不继承 history
Review	任务后单次 review	两阶段 (spec compliance + code quality)
Human-in-loop	任务间可能停	continuous execution · 不停

适用条件（决策树）：

```
有 plan? → 是
任务相互独立? → 是
留在当前 session? → 是
  → 用 subagent-driven-development

任务紧耦合? → 用 executing-plans (开新 session)
```

> 洞察：**Continuous execution + 两阶段 review** 这个组合 = 实战版 LangGraph subgraph + Reviewer 团评审。这就是答疑 §2.6/2.7 讲的“子图 + ReAct”在编码侧的形态。

§2.8 `verification-before-completion` 别说“应该好了”

官方 description：

> "Use when about to claim work is complete, fixed, or passing, **before committing or creating PRs** — requires running verification commands and confirming output before making any success claims; **evidence before assertions always**"

核心信念：

> Claiming work is complete without verification is **dishonesty, not efficiency**.

The Iron Law：

```
NO COMPLETION CLAIMS WITHOUT FRESH VERIFICATION EVIDENCE
```

"If you haven't run the verification command **in this message**, you cannot claim it passes." 历史 message 跑过的不算。

典型失误：

你的声明	真正需要	不够的证据
Tests pass	测试命令输出 0 failures	上一次跑的 / "应该过"
Linter clean	linter 输出 0 errors	部分检查 / 推测
Build succeeds	构建命令 exit 0	linter 过了 / log 看着行
Bug fixed	跑原始 symptom 测试 · 通过	改了代码 / 假设修好了
Regression test works	red-green 闭环验证过	测试跑过一次
Agent completed	VCS diff 显示有变更	agent 自己说"成功"
Requirements met	逐行 checklist	测试通过了

Red Flags • STOP：

- 用 "should" / "probably" / "seems to"
- 验证之前表达满意（"Great!"、"Perfect!"、"Done!"）
- 没验证就要 commit / push / PR
- 信任 agent 的成功报告
- 只做了部分验证
- 觉得“就这一次“
- 累了想结束工作
- **任何暗示成功但其实没跑验证的措辞**

反合理化对照表：

借口	真相
"Should work now"	跑验证
"I'm confident"	信心 ≠ 证据
"Just this once"	没有例外
"Linter passed"	linter ≠ compiler
"Agent said success"	自己独立验证
"I'm tired"	累 ≠ 借口

> 洞察：**这是防止“AI 谎报”的最强 skill**，LLM 默认倾向“looks good”收尾，强制它跑命令、读 exit code、count failures，才能压住“幻觉式完成”。

§2.9 `using-superpowers` 元 skill 控制 skill 调用

官方 description：

> "Use when starting any conversation - establishes how to find and use skills, requiring Skill tool invocation **before ANY response including clarifying questions**"

最强约束语：

```
<EXTREMELY-IMPORTANT>
If you think there is even a 1% chance a skill might apply to what you a
you ABSOLUTELY MUST invoke the skill.

IF A SKILL APPLIES TO YOUR TASK, YOU DO NOT HAVE A CHOICE. YOU MUST USE

This is not negotiable. This is not optional.
You cannot rationalize your way out of this.
</EXTREMELY-IMPORTANT>
```

优先级：

> "1. **User's explicit instructions** (CLAUDE.md, GEMINI.md, AGENTS.md, direct requests) — highest priority
> 2. **Superpowers skills** — override default system behavior where they conflict
> 3. **Default system prompt** — lowest priority"

注意：**用户指令永远赢过 Superpowers**。如果你在CLAUDE.md 要求“不要 TDD” ，skill 写“always TDD” ，AI 听用户的。

§2.10 8 个 skill 的配合关系

```
┌──────────────────────────────────────────────────────────────┐
│  using-superpowers (元 skill · 永远跑在最前面)                  │
└──────────────────────────────────────────────────────────────┘
                        │
                        ▼
                ┌────────────────┐
                │ brainstorming  │  HARD-GATE: 没 design 别动手
                │ (出 design.md) │
                └────────────────┘
                        │ 终点 = 调 writing-plans
                        ▼
                ┌────────────────┐
                │  writing-plans │  bite-sized · TDD · 频繁 commit
                │  (出 plan.md)  │
                └────────────────┘
                        │ 必须声明用谁执行
              ┌─────────────────────┐
              ▼                     ▼
    ┌────────────────┐   ┌──────────────────────────┐
    │ executing-plans │   │ subagent-driven-development  │
    │ (单 session 跑) │   │ (fresh subagent per task)    │
    └────────────────┘   └──────────────────────────┘
              │                     │
              │                     │
              │ 每 task 跑完          │ implementer + spec reviewer
              │                     │   + code quality reviewer
              ▼                     ▼
    ┌──────────────────────────────────────────┐
```

```
    │ test-driven-development（写代码时穿插）      │  IRON LAW · 失败测试先行
    │ systematic-debugging（出 bug 时切换）       │  IRON LAW · 找根因不打补丁
    │ requesting/receiving-code-review（review）  │  fresh subagent · 不演戏
    │ verification-before-completion（claim 前）  │  IRON LAW · 证据先于断言
    └──────────────────────────────────────────┘
```

8 个 skill 的共同设计哲学

读完这 8 个 SKILL.md，能看出 Superpowers 团队的 4 个核心设计模式。

1. **Iron Law + ASCII 框**：每个关键 skill 顶上写一条 IRON LAW 不允许讨论
2. **Anti-Pattern 显式列出**：把 LLM 最容易合理化的退路提前堵死（"This is too simple"、"Just this once"、"Should work"）
3. **Forbidden 禁语**："You're absolutely right!"、"Great!" 这种社交话术明确禁止
4. **Skill 之间链式调用**，brainstorming → writing-plans → executing-plans，每个 skill 终点指向下一个，不靠人记

这套就是 SDD 在日常编码里的精装版。你已经会写 spec / plan / tasks，Superpowers 给的是把这套方法论"封装成 skill + 加上 Iron Law"的工程化实现。

> **写自己 skill 时可以参考的 4 个技巧**：Iron Law 顶上钉、Anti-Pattern 列清单、Forbidden 话术明令禁止、链式 skill 终点声明。

§3 两条实操路径

路径 A • Greenfield（新项目从零开始）

适合：**想拿 Superpowers 起一个新项目练手。**
大致流程：

```
1. 找一个新点子（比如一个 RSS 订阅器 / 个人 Todo App / 周报生成器）
   ↓
2. 在空目录里 / 启动 Claude Code（或 OpenCode）
   ↓
3. 装好 Superpowers 后，让它走 brainstorming skill 跟你聊
   - 它会问你：目标用户？核心场景？技术栈倾向？数据持久化方式？
   - 输出：一份 design.md
   ↓
4. 让 writing-plans 把 design 拆成可执行的任务清单
   - 每个任务 2-5 分钟，含文件路径 + 完整代码片段
   ↓
5. 让 executing-plans 一个一个跑
   - 每跑完一个验证一下（lint / test / 手动跑）
   ↓
6. 出活儿
```

期望：跟以往"上来就让 AI 写代码"完全不同——前 30% 时间花在 brainstorm + plan，后 70% 跑得很快很稳。
预警：

- brainstorm 阶段会问得特别细，**别嫌烦**——这就是它的价值
- plan 阶段如果 AI 给你的步骤太大块（>10 分钟才能做完），是它没拆够细，让它再拆
- execute 阶段如果跳步，立刻打断、回到 plan

参考：obra/superpowers README · 0-to-1 mode + 使用 Superpowers 的元 skill

路径 B • Brownfield（V4 项目接入）

适合：**把 Superpowers 接进 v4-production，用它来做后续的功能加法（评估、订阅、安全加固）。**
大致流程：

```
1. 进 v4-production 目录
   cd ~/ai-knowledge-base/v4-production
   ↓
2. 在 Claude Code / OpenCode 里装 Superpowers（同上 §1）
   ↓
3. 让它先 read 一遍 V4 项目（AGENTS.md / workflows/ / pipeline/ 关键文件）
   ↓
4. 用 brainstorming skill 提你想加的功能：
   - 例：加用户订阅 → "用户能在 Telegram 发 /subscribe rag, bot 记下来，每天 da
   ↓
5. 让 writing-plans 基于 V4 现有结构出 plan
   - 关键：plan 必须 fit V4 的 .opencode/skills/ + workflows/ 既有架构
   ↓
6. executing-plans 跑实现
   ↓
7. 跑完 V4 已有的 eval_test.py 验证没破坏现有功能
```

期望：Superpowers 会先问你"V4 现有架构是啥"，**别偷懒**，把 AGENTS.md / 关键 workflow 文件给它。它读了之后给的 plan 才不会改坏 V3 的 LangGraph 结构。
预警：

- Brownfield 模式下 AI 容易"想推倒重来"，brainstorm 阶段你要明确说："**只增量加，不动 V3 workflows / 不改 OpenClaw skill**"
- plan 出来如果跨了 V4 的边界（比如要重写 organizer），打回让它收敛

Superpowers 官方关于 brownfield 的讨论：issue #739 · How does superpowers work with brownfield project?

> **咖哥建议**：**先做一遍 Greenfield，再做 Brownfield。直接上 Brownfield 容易踩坑**，因为你既不熟悉 Superpowers 的节奏，又要给它划 V4 的边界，挑战可能会比较大。

§4 资源汇总

主要入口

- 📦 obra/superpowers GitHub 主仓库（40.9k stars · 主要文档 / 例子 / issues ）
- 🏪 Claude Code 官方 marketplace 页面（一键安装）

- 🌐 Anthropic 官方 Skills 仓库（Skill 标准 + 官方维护的 base skills）

进阶阅读

- 📘 Spec Kit vs Superpowers 对比 + 组合用法（**强烈推荐**）
- 📝 Superpowers 作者博客 · 我怎么用 coding agents（创作思路）
- 🎓 Superpowers Complete Guide 2026（社区写的实操教程）
- ⭐ Best Claude Code Skills to Try in 2026（生态全景）

跨平台支持

- OpenCode 平台接入文档
- skills.sh · Superpowers 索引页（每个 skill 一句话描述，挑感兴趣的看）

加油！任何SDD后续探讨我们群里面随时讨论。等我把Agent设计模式之类专栏完成了之后，会继续回来深耕SDD的。